

Technical Project Overview

LeGrad-Inspired Temporal Explanations for Speech Emotion Recognition

gradient_based_speech_xai

June 7, 2026

1 Project Overview

This project compares two approaches for explaining a pretrained speech emotion recognition classifier. SpeechXAI provides human-readable explanations based on word-aligned audio segments. The proposed LeGrad-inspired method will derive temporal relevance from gradients associated with the attention mechanisms of a wav2vec 2.0 encoder. The classification task uses speech samples and emotion annotations from IEMOCAP.

The quantitative evaluation uses deletion by silence masking. Important audio regions are replaced by silence while the original signal length is preserved, and the resulting confidence drop for the classifier’s original predicted class is measured. The central methodological decision is to compare word-level SpeechXAI explanations with time-bin or token-level LeGrad explanations under the same total occluded duration.

2 Sources and External References

- **Dataset: IEMOCAP.** The local data follows the five-session IEMOCAP release from the USC Signal Analysis and Interpretation Laboratory. The corpus contains speech, transcripts, and categorical emotion annotations. Project source: [USC IEMOCAP website](#). Dataset paper: [Busso et al., “IEMOCAP: Interactive Emotional Dyadic Motion Capture”](#).
- **Pretrained classifier.** The project uses [superb/wav2vec2-base-superb-er](#), the Hugging Face wav2vec 2.0 emotion-recognition checkpoint loaded by the SpeechXAI repository for IEMOCAP. The model exposes four output classes: anger, happiness/excitement, neutral, and sadness.
- **wav2vec 2.0.** The encoder architecture originates from Baevski et al., [“wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”](#).
- **SUPERB.** The active checkpoint follows the SUPERB speech-processing benchmark described by Yang et al., [“SUPERB: Speech Processing Universal PERFORMANCE Benchmark”](#).
- **SpeechXAI.** The word-level explanation method follows Pastor et al., [“Explaining Speech Classification Models via Word-Level Audio Segments and Paralinguistic Features,” EACL 2024](#). Repository: [elianap/SpeechXAI](#).
- **WhisperX alignment.** SpeechXAI obtains word timestamps with Bain et al., [“WhisperX: Time-Accurate Speech Transcription of Long-Form Audio,” Interspeech 2023](#). WhisperX combines VAD-guided segmentation, Whisper transcription, and external phoneme alignment.

- **LeGrad.** The proposed temporal method is inspired by Boussselham et al., “[LeGrad: An Explainability Method for Vision Transformers via Feature Formation Sensitivity](#)”. Repository: [WalBouss/LeGrad](#).

The current project configuration uses IEMOCAP. No separate YAMLCap dataset source is configured.

3 Model Architecture Used by the Current Pipeline

The executable transformer-relevance scripts in the current branch load the SUPERB HuBERT emotion-recognition checkpoint through `load_hubert_emotion_model`:

```
superb/hubert-base-superb-er
```

The older project configuration still contains references to the wav2vec 2.0 checkpoint used in the SpeechXAI comparison path, but the implemented rollout and head-relevance pipeline is written around a HuBERT sequence-classification model. Notebook `notebooks/00_model_used.ipynb` documents this architecture and provides executable inspection cells.

The executed notebook reports the following active configuration values:

- model type: `hubert`
- architecture: `HubertForSequenceClassification`
- Transformer depth: 12 layers and 12 attention heads
- hidden size: 768
- classifier projection size: 256
- weighted layer aggregation: `use_weighted_layer_sum=True`
- output labels: `neu`, `hap`, `ang`, and `sad`

The notebook also shows that the older YAML configuration still points to the following wav2vec 2.0 checkpoint, while the current transformer-relevance scripts use the HuBERT checkpoint above:

```
superb/wav2vec2-base-superb-er
```

The HuBERT model first converts the waveform into a lower-rate sequence of acoustic frames with a convolutional feature encoder. These frames are then processed by the HuBERT Transformer encoder, which produces temporal hidden states rather than image patches. The convolutional frontend is important for mapping token relevance back to approximate waveform intervals, but it is not the main source of difficulty for adapting LeGrad.

The more important issue is the sequence-classification head. In the Hugging Face HuBERT sequence-classification architecture, the classifier path can be written schematically as follows:

```

waveform
-> convolutional feature encoder
-> HuBERT Transformer hidden states
-> optional learned weighted sum of hidden layers

```

```

-> projector
-> temporal mean pooling
-> classifier
-> emotion logits

```

When `use_weighted_layer_sum` is enabled, the head does not consume one individual Transformer layer directly. Instead, it forms a learned mixture of hidden layers,

$$H_{\text{mix}} = \sum_{l=0}^{12} \text{softmax}(\alpha)_l H_l,$$

and passes this mixed representation into the projection, pooling, and classifier stack. This matters for LeGrad because the original method defines a target score at each Transformer layer before differentiating that score with respect to the attention map of the same layer.

More explicitly, the HuBERT forward pass returns a tuple of hidden-state tensors:

```
outputs.hidden_states = [H_0, H_1, ..., H_12]
```

Here H_0 is the temporal representation entering the Transformer stack after the convolutional/frontend and feature-projection stages, while $H_1, \dots, H_{12}$ are the outputs of the 12 Transformer layers. Each hidden state has shape

$$\text{shape}(H_l) = B \times T \times 768,$$

where B is the batch size, T is the number of HuBERT temporal frames, and 768 is the hidden dimension. The model stacks these tensors along a new layer axis:

```
stacked_hidden_states = stack([H_0, H_1, ..., H_12])
# shape: [B, 13, T, 768]
```

The checkpoint contains one learned scalar per entry in this stack:

```
layer_weights # shape: [13]
```

These raw scalars are the stored trainable parameters. During each forward pass, the model normalizes them with a softmax and uses the resulting coefficients to form the representation that the classification head actually receives:

```

norm_weights = softmax(layer_weights)           # shape: [13]

H_mix = sum_l norm_weights[l] * H_l             # shape: [B, T, 768]

projected = projector(H_mix)                    # shape: [B, T, 256]
pooled = mean_over_time(projected)              # shape: [B, 256]
logits = classifier(pooled)                     # shape: [B, 4]

```

The softmax coefficients are therefore not stored as independent checkpoint parameters. They are recomputed from `layer_weights` every time the model runs, which preserves the same mixture logic if the model is fine-tuned: fine-tuning changes the raw `layer_weights`, and the forward pass then produces updated normalized coefficients.

For explainability, it is important not to confuse these coefficients with a LeGrad layer score. The coefficient $\alpha_l = \text{softmax}(\text{layer_weights})_l$ says how much the trained HuBERT head mixes representation H_l into H_{mix} . It is a learned, forward-time mixture coefficient. It is not by itself a class-dependent score for the current utterance. LeGrad, however, requires a scalar activation whose gradient can be taken with respect to an attention map, for example

$$s_l = \text{classifier}(\text{pool}(\text{projector}(H_l)))_c$$

or the real final score

$$s = \text{classifier}(\text{pool}(\text{projector}(H_{\text{mix}})))_c.$$

If one used only α_l as the score, its derivative with respect to the attention map A_l would not provide an utterance- or class-specific temporal explanation, because α_l is not produced from the current attention map. The learned coefficients are therefore better viewed as possible weights for aggregating already-computed layer relevance maps, not as the relevance score from which those maps are derived.

Thus the learned mixture occurs before the projector and before temporal pooling. The temporal structure is still present in H_{mix} because the tensor keeps the T frame axis; the utterance-level compression happens after projection, when the model averages over time.

The `projector` should therefore not be confused with the layer-mixing operation. The layer mixture uses `layer_weights` to combine $H_0, \dots, H_{12}$ into H_{mix} . The projector is a separate learned linear layer, with `projector.weight` shape (256, 768), applied to each temporal frame of H_{mix} to map 768-dimensional HuBERT features into the 256-dimensional space consumed by the final classifier.

The executed checkpoint-tensor inspection in Notebook `notebooks/00_model_used.ipynb` confirms that `layer_weights` has shape (13,), while the classifier head has `projector.weight` shape (256, 768) and `classifier.weight` shape (4, 256). The 13 mixture entries correspond to H_0 plus the 12 Transformer layer outputs. This is important: the final HuBERT emotion head mixes representations from all these levels before classification, whereas attention maps are only available for the 12 Transformer layers. The forward-time softmax coefficients computed from the inspected raw checkpoint weights are spread across layers rather than concentrated in one layer; they range from 0.067804 for H_1 to 0.084699 for H_{10} .

The notebook’s executed source-code inspection confirms this exact forward path in the installed `transformers` implementation: the model sets `output_hidden_states=True` when weighted layer aggregation is active, stacks all hidden states, applies `softmax(self.layer_weights)`, sums the weighted layers, then applies `self.projector`, temporal pooling, and `self.classifier`. Thus the ordinary classifier score is produced from the learned layer mixture, not from an isolated intermediate layer.

In the ViT/CLIP setting used by LeGrad, the layer-wise score is more direct. Each Transformer block produces visual patch tokens in the same representation stream. The official LeGrad implementation takes an intermediate block output, aggregates the tokens into an image feature, applies the final visual normalization/projection, and computes the target image-text score. Thus the intermediate score follows a natural visual-token pathway:

```
ViT layer H_1
-> token aggregation
-> final visual projection
-> target score_1
```

For HuBERT, applying the classifier head to one individual layer,

HuBERT layer H_1
 -> projector
 -> temporal mean pooling
 -> classifier
 -> target score_1

is a methodological adaptation. It may be useful and testable, but it may not exactly match the representation pathway learned by a checkpoint whose head was trained on H_{mix} rather than on each single H_l . This is the specific architectural reason why a faithful temporal LeGrad implementation for HuBERT is less trivial than applying the original ViT code unchanged.

4 Implemented Methods and Core Logic

The current branch implements a lightweight temporal-relevance pipeline for a HuBERT-based speech emotion classifier. Rather than attempting full Transformer-aware relevance propagation, the implementation uses a compact approximation that combines two ideas: attention flow through the encoder and class-sensitive evidence from the final classifier head.

After checking the official LeGrad implementation, the terminology should be kept precise. Original LeGrad uses the positive gradient of the target activation with respect to an attention map as the explanation signal, $\max(0, \partial s / \partial A)$. It does not multiply that gradient by the attention values, and it does not perform attention rollout. The implementation in this repository should therefore be described as a LeGrad-inspired attention-gradient rollout variant, not as a direct implementation of LeGrad.

The pipeline has three conceptual stages.

1. **Attention-gradient rollout.** The model is run once with attention outputs enabled and gradients are taken with respect to the target class logit. For each Transformer layer, positive gradient-weighted attention is averaged over heads and then propagated through the layer stack to form a joint rollout matrix describing how information can diffuse across tokens.
2. **Head/pooling relevance.** The final HuBERT token representations that enter the classification head are scored for a target class using the classifier weight. Under mean pooling, each token receives a contribution proportional to $H_t \cdot w_c / T$, where H_t is the token representation, w_c is the target-class weight, and T is the number of tokens (or the number of valid tokens for masked pooling).
3. **Method-specific combination.** The rollout signal and the head relevance signal are combined in different ways to produce several explanation modes. These are the methods currently implemented in the branch.

The current implementation does not merely list these variants as names. Each method is produced by the same score-construction function, so the exact relationship between the variants is explicit. The implementation first builds shared intermediate tensors and then chooses a different final combination rule.

1. **Shared rollout construction.** The matrices passed into rollout are not raw attention maps from a single head. They are gradient-weighted layer attention matrices built in `extract_gradient_weighted attentions`. During the forward pass, the model returns one attention tensor per Transformer layer with shape `[batch, heads, tokens, tokens]`. The

target-class logit is then backpropagated, which gives a gradient tensor of the same shape for each attention tensor. For each layer l and head h , the implementation multiplies attention by its target-logit gradient and keeps only positive contributions: $G_{l,h} = \max(0, \nabla A_{l,h} \odot A_{l,h})$. The layer matrix used by rollout is the mean over heads, $A_l = \frac{1}{H} \sum_h G_{l,h}$. Because the attention maps are softmax probabilities and are non-negative, this is equivalent to $\max(0, \nabla A_{l,h}) \odot A_{l,h}$. This is the key methodological difference from original LeGrad, which reduces $\max(0, \nabla A_{l,h})$ directly without multiplying by $A_{l,h}$.

```
def build_gradient_weighted_layer_attentions(attentions, target_logit):
    retain_grad(attentions)
    target_logit.backward()

    layer_attentions = []
    for attn in attentions:
        grad = attn.grad

        weighted = grad * attn
        weighted = clamp_min(weighted, 0)

        # [batch, heads, tokens, tokens] -> [batch, tokens, tokens]
        A_l = mean(weighted, axis=heads)
        layer_attentions.append(A_l)

    return layer_attentions
```

These per-layer matrices are passed to `compute_rollout_attention`. For each layer, the implementation adds an identity matrix to preserve residual/self paths, row-normalizes the result, and then multiplies the matrices through the Transformer stack. If I is the identity matrix, the augmented matrix is $\tilde{A}_l = \text{row_normalize}(A_l + I)$. This row normalization is not a softmax; it simply rescales each row to sum to one after the self-connection has been added. This rollout step is also not part of original LeGrad; it is the attention-flow component of this repository’s hybrid explanation. The joint rollout is then computed by repeated batch matrix multiplication: $R = \tilde{A}_L \tilde{A}_{L-1} \cdots \tilde{A}_1$.

```
def compute_rollout_attention(layer_attentions):
    eye = identity(num_tokens)
    augmented = []

    for A in layer_attentions:
        A_residual = A + eye
        A_residual = A_residual / row_sum(A_residual)
        augmented.append(A_residual)

    R = augmented[0]
    for A in augmented[1:]:
        R = batch_matmul(A, R)

    return R
```

The pure rollout score is obtained by averaging over the final-token rows of the rollout matrix. Because HuBERT sequence classifiers do not use a ViT-like class token, the mean strategy treats all final tokens as possible evidence carriers. The averaged score is min-max scaled by `rollout_to_temporal_relevance` and then normalized so each utterance has a valid distribution over time tokens.

```
raw_rollout = mean(R, axis=final_token_rows)
raw_rollout = min_max_scale(raw_rollout)
rollout_score = normalize_nonnegative(raw_rollout)
```

2. **Target-class head relevance.** The implementation captures the HuBERT hidden states immediately before the classification head. For the current SUPERB HuBERT classifier, the head is linear projection followed by mean pooling and a linear classifier. The code folds the projector into the classifier weight, so the effective class weight in HuBERT hidden-state space is $W_{\text{eff}} = W_{\text{classifier}} W_{\text{projector}}$. For target class c , token t receives the time-dependent logit contribution $H_t \cdot W_{\text{eff},c}/T$, where T is the number of valid tokens. Positive evidence is normalized first; if no positive evidence exists, the implementation falls back to absolute contribution, and if all contributions are zero it uses a uniform distribution over valid tokens.

```
def compute_head_relevance(H, logits, target_class, model, token_mask):
    W_eff = classifier.weight @ projector.weight
    w_target = W_eff[target_class]

    contribution = dot(H, w_target)          # [batch, time]
    contribution = contribution / valid_token_count(token_mask)

    positive = clamp_min(contribution, 0)
    absolute = abs(contribution)

    if sum(positive) > eps:
        return positive / sum(positive)
    if sum(absolute) > eps:
        return absolute / sum(absolute)
    return uniform_over_valid_tokens(token_mask)
```

3. **Contrastive head relevance.** The contrastive branch explains why the target class beats a competing class. If a contrast class is not supplied, the implementation chooses the highest-logit non-target class for each sample. It then replaces the target weight with the margin weight $W_{\text{eff},c} - W_{\text{eff},k}$ and normalizes the resulting token contributions with the same positive/absolute/uniform fallback.

```
def compute_contrastive_head_relevance(H, logits, target_class):
    contrast_class = argmax_non_target_logit(logits, target_class)
    W_eff = classifier.weight @ projector.weight

    margin_weight = W_eff[target_class] - W_eff[contrast_class]
    contribution = dot(H, margin_weight)
```

```

    contribution = contribution / valid_token_count(token_mask)

    return normalize_positive_abs_or_uniform(contribution)

```

Shared inputs used by the explanation methods. After the rollout and head-relevance steps above, the implementation has four named tensors that are reused by the individual explanation methods.

`joint_attention` is the full rollout matrix R returned by `compute_rollout_attention`. It has shape `[batch, tokens, tokens]`. It keeps token-to-token rollout paths: an entry $R_{i,j}$ describes how much final token i routes relevance to token j through the stacked, gradient-weighted attention maps.

`rollout_score` is the one-dimensional temporal rollout score derived from `joint_attention`. It averages the rollout matrix over final-token rows, min-max scales that vector, clamps negative values, and normalizes each utterance so the token scores sum to one. It has shape `[batch, tokens]`.

`head_relevance` is the normalized target-class classifier-head relevance. It is computed from the hidden states entering the head and the effective target-class weight $W_{\text{eff},c}$. It has shape `[batch, tokens]` and gives the tokens that positively support the target class at the classifier head, with absolute/uniform fallbacks when positive evidence is unavailable.

`contrastive_head` is the normalized contrastive classifier-head relevance. It is computed in the same space as `head_relevance`, but uses the target-versus-competitor margin weight $W_{\text{eff},c} - W_{\text{eff},k}$. If the competitor is not specified, the highest-logit non-target class is used. It also has shape `[batch, tokens]`.

```

joint_attention = compute_rollout_attention(grad attentions)

raw_rollout = rollout_to_temporal_relevance(
    joint_attention,
    strategy="mean",
)
rollout_score = normalize_token_scores(raw_rollout)

head_relevance = normalize_token_scores(
    compute_head_relevance(H, logits, target_class, model, token_mask)
)

contrastive_head, contrast_classes = compute_contrastive_head_relevance(
    H,
    logits,
    target_class,
    model,
    contrast_class,
    token_mask,
)
contrastive_head = normalize_token_scores(contrastive_head)

```

Explanation methods.

1. **rollout**. This method returns the normalized rollout distribution directly. It is the target-agnostic baseline: it describes where information can flow through the Transformer, but it does not use the classifier head to ask which tokens support a particular emotion class.

```
scores["rollout"] = rollout_score
```

2. **level3**. This method keeps the original product-style Level 3 formulation used in the branch. It intersects the target-agnostic rollout map with target-class evidence from the classifier head. A token is therefore important only when it is both reachable through the rollout graph and positively contributes to the target class at the head.

```
scores["level3"] = normalize_nonnegative(
    rollout_score * head_relevance
)
```

3. **level3-contrastive**. This variant uses the same product rule, but replaces target-class head relevance with target-versus-competitor margin relevance. It highlights tokens that support the target class relative to the selected competing class, rather than tokens that merely support the target logit in isolation.

```
scores["level3-contrastive"] = normalize_nonnegative(
    rollout_score * contrastive_head
)
```

4. **head-conditioned-rollout**. This method does not average every row of the joint rollout matrix. Instead, it uses the normalized target-class head relevance as a seed distribution over final tokens and propagates that seed backward through the joint rollout matrix. If $R_{i,j}$ describes the rollout path from final token i to input token j , the conditioned score for token j is $\sum_i s_i R_{i,j}$, where s_i is the normalized head-relevance seed.

```
def rollout_from_token_relevance(R, token_relevance):
    seeds = clamp_min(token_relevance, 0)
    seeds = normalize_or_uniform(seeds)

    conditioned = batch_matmul(seeds[:, None, :], R)
    conditioned = squeeze(conditioned, axis=1)
    conditioned = clamp_min(conditioned, 0)

    return conditioned / sum(conditioned)

scores["head-conditioned-rollout"] = rollout_from_token_relevance(
    R,
    head_relevance,
)
```

5. **contrastive-conditioned-rollout**. This is the contrastive version of the conditioned rollout method. The seed distribution is the target-versus-competitor head relevance, and the rollout matrix propagates those contrastive seeds through the Transformer attention paths.

```

scores["contrastive-conditioned-rollout"] = rollout_from_token_relevance(
    R,
    contrastive_head,
)

```

The implementation is therefore a lightweight, auditable hybrid of attention-gradient weighting, rollout, and classifier-head relevance. It is useful for experiments because it is easy to compute, easy to compare across methods, and directly produces a temporal relevance curve over the input waveform. It should not be reported as an exact reproduction of the LeGrad algorithm.

5 Notebook-by-Notebook Technical Overview

5.1 Notebook 00 — Model Used and Classification Head Architecture

Purpose

Document the active model architecture used by the current transformer-relevance pipeline and explain why the classification head design matters for a faithful LeGrad adaptation.

Technical idea

The notebook loads the model configuration for `superb/hubert-base-superb-er`, displays the relevant architecture fields, renders the HuBERT sequence-classification path, and prints the installed `transformers` forward-code excerpt where `use_weighted_layer_sum`, `layer_weights`, `projector`, temporal pooling, and `classifier` appear. It then compares this design with the ViT/CLIP path used by the official LeGrad code.

Decisions made

- The notebook treats `superb/hubert-base-superb-er` as the active model for the current relevance scripts, while displaying the older YAML model source for traceability.
- The notebook inspects the real checkpoint tensor file to report learned `layer_weights` and the concrete projector/classifier parameter shapes without relying only on configuration meta-data.
- The notebook separates the convolutional frontend issue from the classification-head issue: the frontend affects token-to-time mapping, while the head affects how a layer-wise LeGrad score should be defined.

Outcome

Notebook `notebooks/00_model_used.ipynb` was executed and now provides a visual explanation of the active model. The run confirmed a `HubertForSequenceClassification` configuration with 12 Transformer layers, 12 attention heads, hidden size 768, classifier projection size 256, four emotion labels, and `use_weighted_layer_sum=True`. It also prints the installed forward-code excerpt showing the learned hidden-layer mixture before projection, pooling, and classification. The checkpoint inspection confirms that the learned mixture uses 13 entries, one for H_0 and one for each of the 12 Transformer outputs, before the 768-to-256 projector and 4-class emotion classifier.

5.2 Notebook 01 — SpeechXAI Single Example

Purpose

Establish a complete SpeechXAI explanation and deletion-evaluation workflow on one IEMOCAP utterance before integrating the LeGrad-inspired method.

Technical idea

SpeechXAI uses WhisperX to transcribe the utterance and align each word with its actual start and end time. The same timestamp objects are passed into SpeechXAI’s leave-one-out explainer, ensuring that each attribution score refers to the word interval shown in the visualization. SpeechXAI measures importance as the confidence change caused by replacing a word region with silence. The paper describes zeroing the samples inside the WhisperX word interval. The released repository silences a larger interval: it starts 100 ms before the WhisperX word start and ends 40 ms after the WhisperX word end. Thus, the attribution assigned to one word is calculated after silencing the word together with these adjacent audio samples.

Decisions made

- A pretrained wav2vec 2.0 emotion classifier is used so the project can focus on explanation quality rather than classifier training.
- One utterance is used to keep this notebook focused on the explanation workflow and visualization.
- The original predicted class is the attribution and deletion target.
- Silence replacement preserves the waveform length.
- SpeechXAI’s WhisperX transcription is the only source of word text and timing; no character-based or uniform timing approximation is used.
- Transcription is performed once, and the returned word segments are reused by the explainer and plot so words, timestamps, and scores remain index-aligned.
- WhisperX’s per-word transcription confidence is retained in the attribution table so uncertain alignments remain visible during interpretation.
- The paper’s Section 2.1 and Appendix B.1 describe zeroing word segments derived from WhisperX timestamps to preserve the original recording length. They do not specify temporal padding around those segments.
- The released repository’s `utils.removal.py` defines `a=100` and `b=40`. For a WhisperX word interval $[t_{\text{start}}, t_{\text{end}}]$, `remove_word` and `remove_specified_words` silence $[t_{\text{start}} - 100 \text{ ms}, t_{\text{end}} + 40 \text{ ms}]$.
- The notebook reproduces the repository implementation rather than silently replacing it with the paper-level interval description.
- The visualization distinguishes two regions: the lexical interval reported by WhisperX and the larger interval actually silenced by the repository.

- A separate overlap diagnostic displays every effective mask on its own row and computes sample-level mask depth. Regions covered by two or more masks are highlighted explicitly.
- Attribution bars use SpeechXAI’s raw confidence-drop values. They are not rescaled to make the largest word equal to one, because that would hide the magnitude of the model response.
- The waveform-loading path used by the project is checked against SpeechXAI’s pydub-loading path. Both must produce the same predicted class and probability vector before the explanation is accepted.
- Word intervals must be chronological, have positive duration, remain inside the audio, and correspond one-to-one with SpeechXAI’s returned features and scores.
- The notebook rejects an utterance when its first aligned word begins before SpeechXAI’s 100 ms pre-word padding. The upstream removal function does not clamp this slice at zero, so accepting such an example could make the displayed mask differ from the perturbation actually evaluated.
- The final evaluation uses the top- k SpeechXAI words to define the shared masking duration.
- Overlapping padded masks are counted by their union duration so the audio-removal budget is not inflated.
- The notebook links the upstream implementation modules beside the attribution code. The LOO explainer calls single-word removal, the removal utility performs waveform replacement, and the faithfulness evaluator invokes multi-word removal. The linked files are `loo_speech_explainer.py`, `utils_removal.py`, and `faithfulness_measures_speech.py`.
- Notebook 01 writes no result files; all results remain visible in the executed notebook.

What was tested

The selected audio was loaded and classified, then transcribed and aligned with SpeechXAI’s WhisperX function. The returned segments were passed directly into SpeechXAI’s leave-one-out explainer and checked against the returned feature order. Timestamp validity, word-score alignment, and the 100 ms first-word safety condition were validated. The original prediction obtained from SpeechXAI’s pydub representation matched the prediction obtained from the waveform shown in the notebook. The first visualization used real word locations, effective silence masks, raw confidence-drop attribution values, and per-word transcription confidence. The second visualization inspected padded removal intervals and computed overlap depth at waveform-sample resolution. Top- k SpeechXAI deletion was compared with repeated random masking using the same union duration.

Outcome

The notebook executed successfully for utterance `Ses01F_impro01_F005`. WhisperX located the first word at 0.594 seconds, preserving the initial silence and later pauses. The returned word-confidence values ranged from 0.760 to 0.980. The classifier predicted anger with confidence 0.8781. SpeechXAI assigned the largest raw confidence drop, 0.4218, to “what’s”. The overlap diagnostic found six distinct regions covered by multiple padded masks, with a maximum depth of two simultaneous masks. The visualizations, attribution table, consistency checks, and shared-duration evaluation all completed successfully.

Remaining issues

The current validation covers one utterance only. WhisperX confidence is useful diagnostic information but is not a guarantee that every boundary is correct. The 100/40 ms expansion is present in the released Python code but is not documented in the paper, so future experiments should state explicitly whether they reproduce the repository behavior or use unpadded paper-level word intervals. Utterances with a word beginning inside the first 100 ms are currently rejected because of the upstream SpeechXAI slicing behavior; supporting them safely would require clamping the library’s perturbation boundary. The pipeline must be run on a controlled multi-audio set before drawing comparative conclusions.

5.3 Notebook 02 — Attention Access Test

Purpose

Determine whether the wav2vec 2.0 Transformer attention maps required by the LeGrad-inspired method are accessible.

Technical idea

The active classifier is a Hugging Face `Wav2Vec2ForSequenceClassification` model. The diagnostic step requests attention outputs directly from its Transformer encoder and inspects their dimensions and numerical behavior.

Decisions made

- Attention access is treated as a required checkpoint before developing LeGrad.
- The Hugging Face classifier is accessed directly, matching the model-loading path used by SpeechXAI.
- Attention tensors should be inspected across layers and heads and visualized for at least one layer-head pair.

What was tested

The configured SUPERB classifier was run on the same IEMOCAP utterance used in Notebook 01 with attention output enabled. The notebook inspected every returned layer, validated finite values, printed tensor dimensions, and visualized one final-layer attention head.

Outcome

Attention access succeeded. The model returned 12 attention tensors, one per Transformer layer, each shaped `[1, 12, 201, 201]` for the selected utterance.

Remaining issues

Notebook 03 must confirm that these returned attention tensors retain usable, nonzero gradients for the original predicted-class logit.

5.4 Notebook 03 — Attention Gradient Test

Purpose

Verify that gradients of a target class score with respect to Transformer attention maps can be obtained.

Technical idea

LeGrad uses sensitivity to attention maps as its explanation signal. For speech, the target quantity is the classifier score for the original predicted emotion, and the required diagnostic is $\partial s_{\hat{y}}/\partial A_l$ for one or more Transformer layers. The later implementation in this repository uses those gradients in a modified way by multiplying them with attention values before rollout.

Decisions made

- The target is the original predicted-class logit or score.
- Returned attention tensors will retain gradients during backpropagation.
- Gradient shape, magnitude, finite values, and nonzero proportion will be checked before attribution aggregation.
- Logits are preferred over probabilities when probability saturation weakens gradients.

What was tested

The notebook scaffold and reusable gradient-retention utilities are present.

Outcome

The conceptual gradient path is defined. A real model forward and backward pass remains to be validated after Notebook 02 succeeds.

Remaining issues

Gradient availability depends on how the wav2vec 2.0 attention implementation returns intermediate tensors. Hooks may be required if returned attentions are detached from the computational graph.

5.5 Notebook 04 — LeGrad-Inspired Temporal Explanation

Purpose

Convert attention-gradient information into a temporal relevance signal over the speech utterance.

Technical idea

For each Transformer layer, positive class-sensitive attention gradients are combined with attention activations, averaged across heads, propagated with rollout, and reduced to token relevance. Token relevance is then mapped to audio time. This is a LeGrad-inspired adaptation rather than the original LeGrad reduction, which uses the positive attention gradient itself as the explanation signal.

Decisions made

- The method is presented as LeGrad-inspired because it adapts the attention-gradient sensitivity principle from vision to speech.
- Relevance uses positive gradients multiplied by attention values before rollout, which differs from original LeGrad.
- Head and layer aggregation use simple means for interpretability and reproducibility.
- Temporal outputs use small fixed-duration bins or token intervals that can participate in duration-matched deletion evaluation.

What was tested

Reusable aggregation and token-to-time utilities have been prepared.

Outcome

The representation and output format are defined. Real relevance curves depend on successful attention-gradient extraction in Notebook 03.

Remaining issues

The reduction direction of the token-token relevance matrix must be selected empirically and kept fixed. Temporal mapping must also account for the wav2vec 2.0 feature encoder stride.

5.6 Notebook 05 — Quantitative Evaluation

Purpose

Run the complete multi-audio comparison in one coherent experiment. This notebook owns batch explanation generation, common interval representation, equal-duration masking, deletion metrics, and aggregate quantitative plots.

Technical idea

The validated SpeechXAI workflow from Notebook 01 and LeGrad-inspired workflow from Notebook 04 are applied to the same controlled IEMOCAP set. SpeechXAI top- k words define a union duration X . SpeechXAI masks, LeGrad-inspired top bins, and repeated random bins are then compared after silencing exactly X seconds while preserving waveform length.

Decisions made

- Batch SpeechXAI generation is part of evaluation rather than a separate notebook because its single-audio behavior is already established in Notebook 01.
- Interval unification and masking are part of the evaluation operation rather than independent experimental outcomes.
- Both explanation methods use the common columns `audio_id`, `start`, `end`, and `score`.
- SpeechXAI is the source of duration X because its native unit is a variable-duration word.

- Top- k values are 1, 2, 3, and 5.
- LeGrad-inspired bins are selected by score until their union duration reaches X .
- Random bins use the same duration and are repeated 20 times.
- The main metric is absolute confidence drop for the original predicted class.
- Relative confidence drop and prediction flip rate are secondary metrics.
- Aggregate tables, deletion curves, boxplots, duration checks, and flip rates remain in this notebook because they are direct outputs of the quantitative experiment.

What was tested

Notebook 01 validated real SpeechXAI attribution, silence masking, union-duration accounting, confidence drop, and repeated same-duration random masking on one utterance. The consolidated Notebook 05 currently contains the protocol scaffold that will extend those validated operations to SpeechXAI and LeGrad across the evaluation set.

Outcome

The evaluation schema and fairness constraints are defined and validated at single-audio scale. The previous standalone SpeechXAI, interval-masking, deletion, and final-summary placeholders have been removed because their responsibilities now belong to this one quantitative experiment.

Remaining issues

The complete run requires a working LeGrad example from Notebook 04, a documented evaluation set, and execution across all selected audios.

5.7 Notebook 06 — Qualitative Examples

Purpose

Create interpretable examples that show where SpeechXAI and LeGrad agree or disagree and how masking changes model confidence.

Technical idea

Each example combines the waveform, word-level SpeechXAI intervals, the LeGrad temporal relevance curve, selected masking regions, and before/after classifier confidence.

Decisions made

- Examples will be selected from quantitative outcomes rather than chosen only for visual appeal.
- The final set should include agreement, LeGrad-favored, SpeechXAI-favored, and difficult cases.
- Interpretations will distinguish model faithfulness from human plausibility.

What was tested

The notebook structure and plotting requirements have been defined.

Outcome

The qualitative analysis plan is ready.

Remaining issues

Representative cases can be selected after Notebook 05 produces the complete deletion results. Aggregate tables remain in Notebook 05; this notebook is reserved for case-level interpretation.

6 Evaluation Design

The final quantitative evaluation follows these steps:

1. SpeechXAI returns importance scores for word-aligned audio segments.
2. For each audio and each $k \in \{1, 2, 3, 5\}$, the top- k SpeechXAI words are selected.
3. The selected words define a total duration X seconds.
4. Those SpeechXAI word intervals are replaced by silence, and the confidence drop for the original predicted class is measured.
5. LeGrad-inspired time bins are ranked by relevance, and the highest-ranked bins whose total duration equals X are selected.
6. Those LeGrad-inspired regions are replaced by silence, and the same confidence drop is measured.
7. Random time bins with total duration X are selected repeatedly to form a stable random baseline.
8. All methods use the same audio, classifier, target class, masking operation, and removed duration.

SpeechXAI naturally explains words, while the LeGrad-inspired method explains Transformer time bins or tokens. Equalizing the total occluded duration makes the comparison focus on where each method assigns relevance.

The main metric is:

$$\text{drop} = p_{\text{original}}(\hat{y}) - p_{\text{masked}}(\hat{y}),$$

where $\hat{y}$ is the class predicted from the original audio. Secondary metrics are relative confidence drop and prediction flip rate. Mean masked duration is reported for every top- k setting.

7 Methodological Decisions Summary

Component	Decision	Reason
Dataset	IEMOCAP speech, transcripts, and emotion labels	It is a standard speech emotion resource and provides the audio and annotations required by the selected classifier.
Classifier	SUPERB wav2vec2-base-superb-er checkpoint used by SpeechXAI	Using the same classifier removes model choice as a confound when comparing SpeechXAI and LeGrad-inspired explanations.
Model label space	Anger, happiness/excitement, neutral, sadness	These are the four outputs supported by the selected pretrained classifier.
Target class	Original predicted class	Faithfulness is evaluated relative to the model decision being explained.
SpeechXAI unit	Word-aligned audio segments	This is SpeechXAI’s native, human-readable explanation unit.
LeGrad unit	Transformer tokens or fixed temporal bins	Attention-gradient relevance is naturally produced at the encoder-token level.
LeGrad-inspired formulation	Positive attention gradients multiplied by attention values, then used in rollout	This borrows LeGrad’s attention-gradient sensitivity but is not the original LeGrad reduction.
Shared duration	X seconds from SpeechXAI top- k words	This makes word-level and time-bin explanations comparable under equal audio removal.
Top- k values	1, 2, 3, and 5 words	These settings provide a compact deletion curve while respecting SpeechXAI’s native output.
Masking method	Silence replacement with unchanged waveform length	It provides a simple perturbation and avoids duration changes that could independently affect the classifier.
Random baseline	Same-duration random bins, repeated 20 times	Repetition estimates baseline variance and provides a stable reference.
Main metric	Confidence drop for the original predicted class	It measures how strongly the selected regions support the model’s original decision.
Secondary metrics	Relative confidence drop and prediction flip rate	They complement absolute confidence drop across samples with different confidence levels.
Notebook 01 scope	One utterance, in-memory outputs	It establishes an understandable baseline and avoids mixing pipeline validation with dataset-scale evaluation.